

FILTRES NUMÉRIQUES

Complément de cours avec Exercices MATLAB

Dr OBONO BIYOBO Arnaud, ENSET, Université de Douala

- ✓ **Rappels** sur le traitement du signal discret
- ✓ **Transformée en Z** : définition, propriétés et exercices
- ✓ **Passage Z $\leftrightarrow$ équation de récurrence** (avec exercices corrigés)
- ✓ **Filtres RIF** (Réponse Impulsionnelle Finie)
- ✓ **Filtres RII** (Réponse Impulsionnelle Infinie)
- ✓ **Stabilité** des filtres numériques
- ✓ **Représentations graphiques** des structures de filtres
- ✓ **Codes MATLAB** fonctionnels et commentés
- ✓ **Exercices pratiques** avec corrections détaillées

💡 GINFO4

Ce cours de filtre numérique est essentiel pour les étudiants de GINFO option TIC et Informatique Industrielle. Il permet de maîtriser le traitement des signaux numériques utilisé dans les télécommunications, les réseaux, les systèmes multimédias, l'automatisation et les systèmes embarqués de l'industrie moderne.

Table des matières

1	Rappels Indispensables	2
1.1	Signal analogique vs signal numérique	2
1.1.1	Paramètres clés	2
1.2	Signaux de base	2
2	La Transformée en Z	3
2.1	Pourquoi la transformée en Z ?	3
2.2	Définition	3
2.3	Transformées usuelles à connaître	4
2.4	Propriétés importantes	4
3	De la Transformée en Z à l'Équation de Récurrence	4
3.1	Fonction de transfert en Z	5
3.2	Passage de $H(z)$ vers l'équation de récurrence	5
3.3	Passage de l'équation de récurrence vers $H(z)$	6
4	Filtres RIF (Réponse Impulsionnelle Finie)	6
4.1	Définition et équation	6
4.2	Fonction de transfert d'un filtre RIF	7
4.3	Structure d'un filtre RIF	7
4.4	Avantages et inconvénients des filtres RIF	7
5	Filtres RII (Réponse Impulsionnelle Infinie)	7
5.1	Définition et équation	8
5.2	Fonction de transfert d'un filtre RII	8
5.3	Structure d'un filtre RII (Forme Directe I)	8
5.4	Comparaison RIF vs RII	9
6	Stabilité des Filtres Numériques	9
6.1	Définition de la stabilité BIBO	10
6.2	Condition de stabilité dans le domaine temporel	10
6.3	Condition de stabilité dans le domaine en Z	10
6.4	Illustration graphique des pôles et zéros	11
6.5	Comment trouver les pôles ?	11
7	Exercices sur la Transformée en Z	12
8	Exercices MATLAB Pratiques	14
8.1	Exercice MATLAB 1 : Exploration de la transformée en Z	14
8.2	Exercice MATLAB 2 : Implémentation d'un filtre RIF	16
8.3	Exercice MATLAB 3 : Conception d'un filtre RII de Butterworth	18
8.4	Exercice MATLAB 4 : Analyse de la stabilité	21
8.5	Exercice MATLAB 5 : Comparaison RIF vs RII	24
9	Récapitulatif et Aide-mémoire	26

1 Rappels Indispensables

1.1 Signal analogique vs signal numérique

📖 Définition : Signal numérique (ou discret)

Un **signal numérique** est un signal défini uniquement à des instants précis et séparés régulièrement dans le temps. On le note $x[n]$ où n est un entier qui représente l'indice temporel.

💡 Remarque

Pensez au son que vous enregistrez avec un microphone : le son dans l'air est un signal analogique (continu). Quand votre ordinateur le capture, il le *échantillonne* — il ne prend une mesure qu'à intervalles réguliers (par exemple 44100 fois par seconde pour la qualité CD). Ce signal échantillonné est un signal numérique !

1.1.1 Paramètres clés

- **Fréquence d'échantillonnage** F_e (en Hz) : nombre de mesures par seconde. Ex : $F_e = 8000$ Hz pour la téléphonie.
- **Période d'échantillonnage** $T_e = 1/F_e$ (en secondes).
- **Théorème de Shannon** : pour capturer correctement un signal, il faut $F_e \geq 2 \cdot f_{max}$, où f_{max} est la fréquence maximale du signal analogique.

1.2 Signaux de base

📖 Définition : Impulsion unité $\delta[n]$

$$\delta[n] = \begin{cases} 1 & \text{si } n = 0 \\ 0 & \text{sinon} \end{cases}$$

C'est le signal le plus court et le plus "piqué" qui existe.

📖 Définition : Échelon unité $u[n]$

$$u[n] = \begin{cases} 1 & \text{si } n \geq 0 \\ 0 & \text{si } n < 0 \end{cases}$$

C'est un signal qui "s'allume" brutalement à $n = 0$ et reste à 1.

2 La Transformée en Z

2.1 Pourquoi la transformée en Z ?

💡 Analogie pédagogique

De la même façon que la transformée de Laplace simplifie les équations différentielles (avec des dérivées) en simples équations algébriques (avec des multiplications), la **transformée en Z** transforme les **équations aux différences** (les équations qui définissent les filtres numériques) en équations algébriques simples. C'est un outil de calcul très puissant !

2.2 Définition

📖 Définition : Transformée en Z bilatérale

Pour un signal $x[n]$, la transformée en Z est définie par :

$$X(z) = \mathcal{Z}\{x[n]\} = \sum_{n=-\infty}^{+\infty} x[n] \cdot z^{-n}$$

où z est une variable **complexe** ($z \in \mathbb{C}$).

📖 Définition : Transformée en Z unilatérale (cas causal)

Pour les signaux causaux (nuls pour $n < 0$) :

$$X(z) = \sum_{n=0}^{+\infty} x[n] \cdot z^{-n}$$

📖 Définition : Région de Convergence (ROC)

La **Région de Convergence** est l'ensemble des valeurs de z pour lesquelles la somme infinie converge (ne "part pas à l'infini"). Elle s'écrit généralement sous la forme $|z| > r$ (extérieur d'un cercle de rayon r).

2.3 Transformées usuelles à connaître

Signal $x[n]$	Transformée $X(z)$	ROC
$\delta[n]$	1	Tout $\mathbb{C}$
$u[n]$	$\frac{z}{z-1} = \frac{1}{1-z^{-1}}$	$ z > 1$
$a^n u[n]$	$\frac{z}{z-a} = \frac{1}{1-az^{-1}}$	$ z > a $
$\delta[n-k]$	z^{-k}	$\mathbb{C} \setminus \{0\}$
$n \cdot a^n u[n]$	$\frac{az^{-1}}{(1-az^{-1})^2}$	$ z > a $

2.4 Propriétés importantes

★ Théorème : Linéarité

$$\mathcal{Z} \{ \alpha x[n] + \beta y[n] \} = \alpha X(z) + \beta Y(z)$$

On peut transformer terme par terme une somme.

★ Théorème : Retard (décalage temporel)

$$\mathcal{Z} \{ x[n-k] \} = z^{-k} \cdot X(z)$$

Un retard de k échantillons correspond à une multiplication par z^{-k} . C'est LA propriété fondamentale qui nous permettra de passer d'une équation de récurrence à une fonction de transfert !

★ Théorème : Convolution

$$\mathcal{Z} \{ x[n] * h[n] \} = X(z) \cdot H(z)$$

La convolution dans le domaine temporel devient une simple multiplication dans le domaine en Z .

3 De la Transformée en Z à l'Équation de Récurrence

💡 Objectif de ce chapitre

C'est un chapitre **essentiel** ! Vous devez être capable de :

1. Partir d'une **fonction de transfert** $H(z)$ et trouver l'**équation de récurrence** correspondante.
2. Partir d'une **équation de récurrence** et trouver la **fonction de transfert** $H(z)$.

3.1 Fonction de transfert en Z

Définition : Fonction de transfert $H(z)$

La **fonction de transfert** d'un filtre numérique est le rapport de la transformée en Z de la sortie sur celle de l'entrée, avec conditions initiales nulles :

$$H(z) = \frac{Y(z)}{X(z)}$$

Elle s'écrit généralement sous forme de fraction rationnelle :

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2} + \dots + b_M z^{-M}}{1 + a_1 z^{-1} + a_2 z^{-2} + \dots + a_N z^{-N}} = \frac{\sum_{k=0}^M b_k z^{-k}}{1 + \sum_{k=1}^N a_k z^{-k}}$$

3.2 Passage de $H(z)$ vers l'équation de récurrence

Méthode à suivre

Pour passer de $H(z)$ à l'équation de récurrence :

1. Écrire $H(z) = Y(z)/X(z)$
2. Faire une **produit en croix** : $Y(z) \cdot (\text{dénominateur}) = X(z) \cdot (\text{numérateur})$
3. **Appliquer la transformée inverse** : remplacer z^{-k} par un retard de k , c'est-à-dire $z^{-k}Y(z) \rightarrow y[n - k]$

Méthode de passage $H(z) \rightarrow$ récurrence

Soit $H(z) = \frac{1 + 2z^{-1}}{1 - 0,5z^{-1}}$

Étape 1 : $\frac{Y(z)}{X(z)} = \frac{1 + 2z^{-1}}{1 - 0,5z^{-1}}$

Étape 2 (produit en croix) :

$$Y(z) \cdot (1 - 0,5z^{-1}) = X(z) \cdot (1 + 2z^{-1})$$

$$Y(z) - 0,5z^{-1}Y(z) = X(z) + 2z^{-1}X(z)$$

Étape 3 (transformée inverse, $z^{-k}Y(z) \rightarrow y[n - k]$) :

$$y[n] = 0,5 \cdot y[n - 1] + x[n] + 2 \cdot x[n - 1]$$

3.3 Passage de l'équation de récurrence vers $H(z)$

! Méthode inverse

Pour passer d'une équation de récurrence à $H(z)$:

1. **Appliquer la transformée en Z** à chaque terme, en utilisant $y[n - k] \rightarrow z^{-k}Y(z)$ et $x[n - k] \rightarrow z^{-k}X(z)$
2. **Regrouper** les termes en $Y(z)$ et $X(z)$
3. **Calculer** $H(z) = Y(z)/X(z)$

✍ Méthode récurrence $\rightarrow H(z)$

Équation de récurrence : $y[n] = 0,8 \cdot y[n - 1] - 0,1 \cdot y[n - 2] + x[n] + x[n - 1]$

Étape 1 : Transformée en Z :

$$Y(z) = 0,8z^{-1}Y(z) - 0,1z^{-2}Y(z) + X(z) + z^{-1}X(z)$$

Étape 2 : Regroupement :

$$Y(z)(1 - 0,8z^{-1} + 0,1z^{-2}) = X(z)(1 + z^{-1})$$

Étape 3 :

$$H(z) = \frac{1 + z^{-1}}{1 - 0,8z^{-1} + 0,1z^{-2}}$$

4 Filtres RIF (Réponse Impulsionnelle Finie)

4.1 Définition et équation

📖 Définition : Filtre RIF

Un **filtre RIF** (ou FIR en anglais pour *Finite Impulse Response*) est un filtre dont la sortie ne dépend que des **valeurs passées et présentes de l'entrée**, jamais de la sortie elle-même.

Son équation de récurrence est :

$$y[n] = \sum_{k=0}^M b_k \cdot x[n - k] = b_0x[n] + b_1x[n - 1] + \dots + b_Mx[n - M]$$

La **réponse impulsionnelle** du filtre RIF est directement :

$$h[n] = b_n \text{ pour } 0 \leq n \leq M, \text{ et } 0 \text{ ailleurs}$$

Elle est donc de durée **finie** ($M + 1$ coefficients).

4.2 Fonction de transfert d'un filtre RIF

$$H(z) = \sum_{k=0}^M b_k z^{-k} = b_0 + b_1 z^{-1} + \dots + b_M z^{-M}$$

💡 Remarque

Un filtre RIF n'a **que des zéros** (pas de pôles sauf à l'origine). Il est **toujours stable** (on ne divise jamais par quelque chose qui pourrait devenir infini).

4.3 Structure d'un filtre RIF

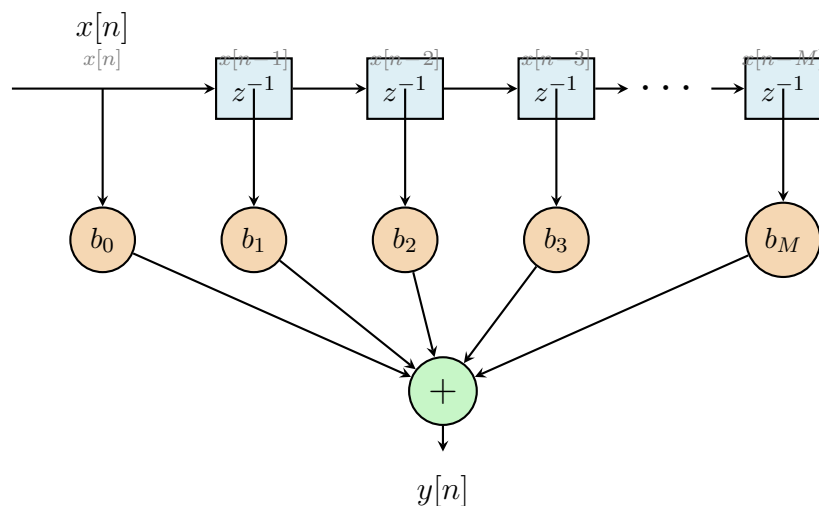


Figure 1 : Structure en forme directe d'un filtre RIF d'ordre M

4.4 Avantages et inconvénients des filtres RIF

Avantages	Inconvénients
Toujours stable	Ordre M souvent très élevé
Peut avoir une phase linéaire exacte	Coût calcul plus élevé
Pas de problèmes de précision numérique	Délai de groupe important
Conception simple et intuitive	

5 Filtres RII (Réponse Impulsionnelle Infinie)

5.1 Définition et équation

Définition : Filtre RII

Un **filtre RII** (ou IIR en anglais pour *Infinite Impulse Response*) est un filtre dont la sortie dépend **à la fois des valeurs passées de l'entrée ET des valeurs passées de la sortie** (présence d'une boucle de rétroaction).

Son équation de récurrence générale est :

$$y[n] = \sum_{k=0}^M b_k \cdot x[n-k] - \sum_{k=1}^N a_k \cdot y[n-k]$$

La **réponse impulsionnelle** est théoriquement infinie (elle ne s'annule jamais complètement, même si elle diminue).

Remarque

La présence du terme $-\sum a_k y[n-k]$ signifie que le filtre "se souvient" de ses propres sorties passées. C'est ce qu'on appelle la **rétroaction** (ou *feedback* en anglais). C'est ce qui peut rendre le filtre instable !

5.2 Fonction de transfert d'un filtre RII

$$H(z) = \frac{B(z)}{A(z)} = \frac{\sum_{k=0}^M b_k z^{-k}}{1 + \sum_{k=1}^N a_k z^{-k}}$$

5.3 Structure d'un filtre RII (Forme Directe I)

La structure **Forme Directe I** sépare la partie directe (coefficients b_k , en bleu) et la partie réursive (coefficients a_k , en rouge). Le signal traverse d'abord la chaîne de retards de l'entrée, puis les résultats sont sommés avec les retours de la sortie.

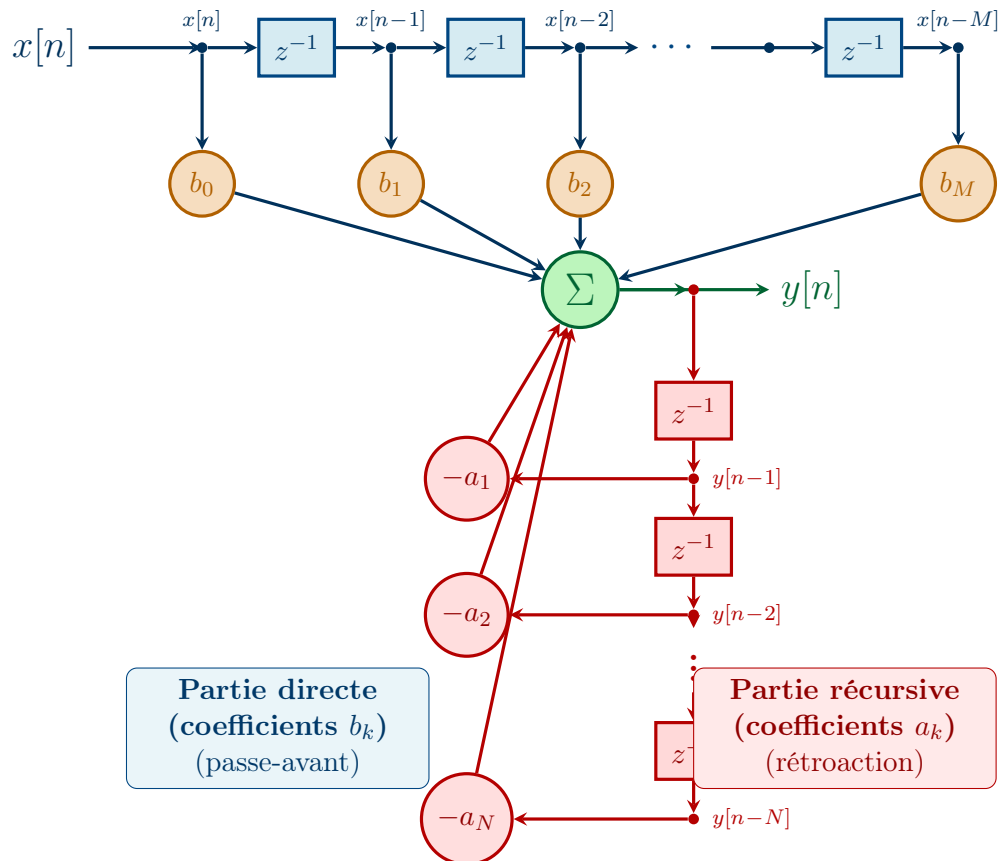


Figure 2 : Structure Forme Directe I d'un filtre RII d'ordre (M, N) — en **bleu** : chaîne de retards de l'entrée ; en **rouge** : boucle de rétroaction de la sortie ; en **vert** : nœud de somme et signal de sortie.

5.4 Comparaison RIF vs RII

Critère	Filtre RIF	Filtre RII
Stabilité	Toujours stable	Peut être instable
Phase	Peut être linéaire	Jamais linéaire
Ordre pour un gabarit	Très élevé	Faible
Rétroaction	Non	Oui
Réponse impulsionnelle	Finie	Infinie
Inspiré de	—	Filtres analogiques

6 Stabilité des Filtres Numériques

6.1 Définition de la stabilité BIBO

📖 Définition : Stabilité BIBO

Un filtre est dit **BIBO stable** (*Bounded-Input Bounded-Output* = Entrée Bornée Sortie Bornée) si pour toute entrée bornée $|x[n]| \leq M_x < \infty$, la sortie reste bornée $|y[n]| \leq M_y < \infty$.

En pratique : si vous "excitez" le filtre avec un signal raisonnable, la sortie ne part pas à l'infini !

6.2 Condition de stabilité dans le domaine temporel

★ Théorème : Condition de stabilité (domaine temporel)

Un filtre est BIBO stable si et seulement si sa réponse impulsionnelle $h[n]$ est **absolument sommable** :

$$\sum_{n=-\infty}^{+\infty} |h[n]| < \infty$$

Pour un filtre RIF, cette condition est **toujours satisfaite** (somme finie).

6.3 Condition de stabilité dans le domaine en Z

★ Théorème : Condition de stabilité (pôles)

Un filtre causal est BIBO stable si et seulement si **tous ses pôles sont strictement à l'intérieur du cercle unité** dans le plan complexe z :

$$\text{Stable} \iff \forall p_i \text{ (pôle)}, |p_i| < 1$$

! Attention !

- Si un pôle est **sur le cercle unité** ($|p| = 1$) : stabilité marginale (oscillations qui ne s'arrêtent pas).
- Si un pôle est **à l'extérieur du cercle unité** ($|p| > 1$) : **filtre instable** (la sortie diverge vers l'infini!).
- Les filtres RIF n'ont pas de pôles (sauf à $z = 0$), donc **toujours stables**.

6.4 Illustration graphique des pôles et zéros

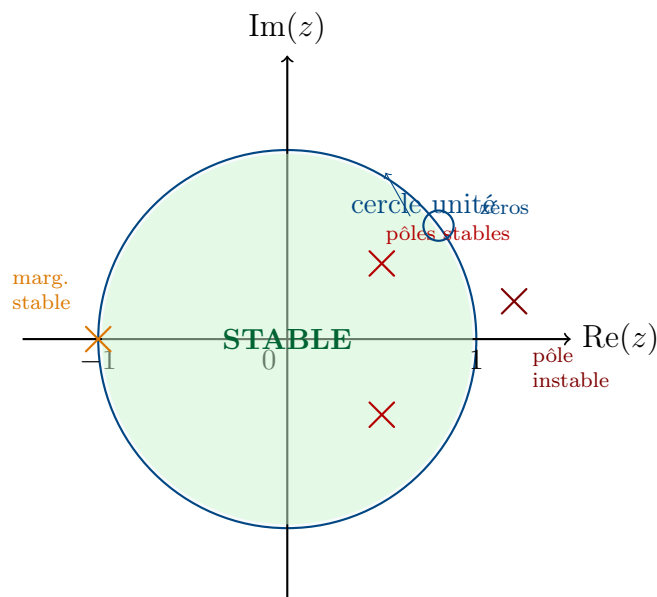


Figure 3 : Plan des pôles et zéros — condition de stabilité

6.5 Comment trouver les pôles ?

! Méthode

Les **pôles** sont les racines du **dénominateur** de $H(z)$.

Les **zéros** sont les racines du **numérateur** de $H(z)$.

Pour $H(z) = \frac{N(z)}{D(z)}$:

- Pôles : solutions de $D(z) = 0$
- Zéros : solutions de $N(z) = 0$

✍ Analyse de stabilité

Soit $H(z) = \frac{z + 1}{z^2 - 0,5z + 0,25}$

Pôles : solutions de $z^2 - 0,5z + 0,25 = 0$

$$z = \frac{0,5 \pm \sqrt{0,25 - 1}}{2} = \frac{0,5 \pm j\sqrt{0,75}}{2} = 0,25 \pm j0,433$$

$$|p| = \sqrt{0,25^2 + 0,433^2} = \sqrt{0,25} = 0,5 < 1$$

Conclusion : Les deux pôles ont un module $< 1 \Rightarrow$ **filtre STABLE**.

7 Exercices sur la Transformée en Z

⚡ Exercice 1 : Transformée en Z d'un signal simple

Calculer la transformée en Z du signal suivant et déterminer sa ROC :

$$x[n] = (0,5)^n u[n] + (-0,3)^n u[n]$$

✓ Correction – Exercice 1

On utilise le tableau des transformées usuelles : $\mathcal{Z}\{a^n u[n]\} = \frac{z}{z-a}$ pour $|z| > |a|$.

Transformée du premier terme ($a = 0,5$) :

$$\mathcal{Z}\{(0,5)^n u[n]\} = \frac{z}{z-0,5} = \frac{1}{1-0,5z^{-1}}, \quad |z| > 0,5$$

Transformée du second terme ($a = -0,3$) :

$$\mathcal{Z}\{(-0,3)^n u[n]\} = \frac{z}{z+0,3} = \frac{1}{1+0,3z^{-1}}, \quad |z| > 0,3$$

Par linéarité :

$$X(z) = \frac{z}{z-0,5} + \frac{z}{z+0,3} = \frac{z(z+0,3) + z(z-0,5)}{(z-0,5)(z+0,3)}$$

$$X(z) = \frac{2z^2 - 0,2z}{z^2 - 0,2z - 0,15}$$

ROC : intersection des deux ROC : $|z| > 0,5$ (la plus restrictive).

⚡ Exercice 2 : De $H(z)$ vers l'équation de récurrence

Soit la fonction de transfert :

$$H(z) = \frac{3 - z^{-1}}{1 + 0,2z^{-1} - 0,24z^{-2}}$$

1. Donner l'équation de récurrence du filtre.
2. Identifier le type (RIF ou RII).
3. Trouver les pôles et analyser la stabilité.

✓ Correction – Exercice 2

1. Équation de récurrence :

$$\frac{Y(z)}{X(z)} = \frac{3 - z^{-1}}{1 + 0,2z^{-1} - 0,24z^{-2}}$$

Produit en croix :

$$Y(z)(1 + 0,2z^{-1} - 0,24z^{-2}) = X(z)(3 - z^{-1})$$

$$Y(z) + 0,2z^{-1}Y(z) - 0,24z^{-2}Y(z) = 3X(z) - z^{-1}X(z)$$

Transformée inverse :

$$y[n] = -0,2y[n-1] + 0,24y[n-2] + 3x[n] - x[n-1]$$

2. Type : Il y a des termes $y[n-k]$ au second membre $\Rightarrow$ **filtre RII**, d'ordre 2.

3. Pôles : Racines de $z^2 + 0,2z - 0,24 = 0$ (après multiplication par z^2) :

$$z = \frac{-0,2 \pm \sqrt{0,04 + 0,96}}{2} = \frac{-0,2 \pm 1}{2}$$

$p_1 = 0,4$ et $p_2 = -0,6$.

$|p_1| = 0,4 < 1$ et $|p_2| = 0,6 < 1 \Rightarrow$ **Filtre STABLE**.

⚡ Exercice 3 : De l'équation de récurrence vers $H(z)$

On donne l'équation de récurrence :

$$y[n] = x[n] + x[n-1] + x[n-2] + x[n-3]$$

1. Trouver $H(z)$.
2. Identifier le type de filtre.
3. Calculer la réponse en fréquence $H(e^{j\omega})$ et interpréter.

✓ Correction – Exercice 3

1. Calcul de $H(z)$:

Transformée en Z : $Y(z) = X(z)(1 + z^{-1} + z^{-2} + z^{-3})$

$$H(z) = 1 + z^{-1} + z^{-2} + z^{-3} = \frac{z^3 + z^2 + z + 1}{z^3}$$

2. Type : Pas de termes $y[n-k]$, seulement des termes en $x \Rightarrow$ **filtre RIF** d'ordre 3 (coefficients $b_k = 1$ pour $k = 0, 1, 2, 3$).

3. Réponse en fréquence : On pose $z = e^{j\omega}$:

$$H(e^{j\omega}) = 1 + e^{-j\omega} + e^{-2j\omega} + e^{-3j\omega} = e^{-j3\omega/2} \cdot \frac{\sin(2\omega)}{\sin(\omega/2)}$$

Ce filtre est un **filtre moyennneur** (il fait la moyenne de 4 échantillons). Il atténue les hautes fréquences : c'est un **filtre passe-bas** !

8 Exercices MATLAB Pratiques

8.1 Exercice MATLAB 1 : Exploration de la transformée en Z

⚡ Exercice MATLAB 1 : Visualisation des pôles et zéros

Analyser et visualiser les pôles et zéros de la fonction de transfert :

$$H(z) = \frac{z^2 + 0,5z}{z^2 - 0,6z + 0,25}$$

Tracer la réponse en fréquence et analyser la stabilité.

🔗 Code MATLAB : Exercice MATLAB 1 – Pôles, zéros et réponse en fréquence

```

1 %%
  =====
2 % EXERCICE 1 : Poles, zeros et reponse en frequence
3 %%
  =====
4 clear all; clc; close all;
5
6 % --- Definition de H(z) avec coefficients ---
7 % H(z) = (z^2 + 0.5z) / (z^2 - 0.6z + 0.25)
8 % En puissances negatives de z :
9 % H(z) = (1 + 0.5*z^-1) / (1 - 0.6*z^-1 + 0.25*z^-2)
10
11 b = [1, 0.5, 0]; % Coefficients du numerateur (b0, b1, b2)
12 a = [1, -0.6, 0.25]; % Coefficients du denominateur (a0, a1,
    a2)
13
14 %--- 1. Diagramme poles-zeros ---
15 figure('Name','Exercice 1 - Poles et zeros','NumberTitle','off
    ');
16
17 subplot(2,2,1);
18 zplane(b, a); % Trace le plan des poles et zeros
19 title('Diagramme Poles-Zeros', 'FontSize', 13, 'FontWeight', '
    bold');
20 grid on;
21
22 % Verifier la stabilite : poles a l'interieur du cercle unite
    ?
23 poles = roots(a);
  
```

```

24 zeros_H = roots(b);
25 disp('=== ANALYSE DE STABILITE ===');
26 disp('Poles du filtre :'); disp(poles);
27 disp('Module des poles :'); disp(abs(poles));
28 if all(abs(poles) < 1)
29     disp('>>> Filtre STABLE : tous les poles sont dans le
        cercle unite');
30 else
31     disp('>>> ATTENTION : Filtre INSTABLE !');
32 end
33
34 %--- 2. Reponse en frequence (magnitude) ---
35 subplot(2,2,2);
36 [H, w] = freqz(b, a, 512); % 512 points, de 0 a pi
37 plot(w/pi, 20*log10(abs(H)), 'b', 'LineWidth', 2);
38 xlabel('Frequence normalisee (\omega/\pi)');
39 ylabel('|H(e^{j\omega})| en dB');
40 title('Reponse en frequence (dB)', 'FontSize', 13, 'FontWeight
    ', 'bold');
41 grid on; xlim([0 1]);
42
43 %--- 3. Phase ---
44 subplot(2,2,3);
45 plot(w/pi, angle(H)*180/pi, 'r', 'LineWidth', 2);
46 xlabel('Frequence normalisee (\omega/\pi)');
47 ylabel('Phase (degres)');
48 title('Reponse en phase', 'FontSize', 13, 'FontWeight', 'bold
    ');
49 grid on; xlim([0 1]);
50
51 %--- 4. Reponse impulsionnelle ---
52 subplot(2,2,4);
53 [h, t] = impz(b, a, 30); % 30 premiers echantillons
54 stem(t, h, 'filled', 'b', 'LineWidth', 1.5);
55 xlabel('Echantillon n');
56 ylabel('h[n]');
57 title('Reponse impulsionnelle', 'FontSize', 13, 'FontWeight',
    'bold');
58 grid on;
59
60 sgtitle('Analyse Complete du Filtre H(z)', 'FontSize', 15, '
    FontWeight', 'bold');

```

✓ Correction – Exercice MATLAB 1

Résultats attendus :

Les pôles de $H(z) = \frac{z^2 + 0,5z}{z^2 - 0,6z + 0,25}$ sont les racines de $z^2 - 0,6z + 0,25 = 0$, soit

$$p_{1,2} = 0,3 \pm j0,4.$$

$$|p| = \sqrt{0,09 + 0,16} = \sqrt{0,25} = 0,5 < 1 : \text{filtre stable.}$$

La réponse en fréquence montre un comportement passe-bande (les pôles proches du cercle unité amplifient une certaine bande de fréquences).

8.2 Exercice MATLAB 2 : Implémentation d'un filtre RIF

⚡ Exercice MATLAB 2 : Filtre RIF moyennneur

1. Concevoir un filtre RIF moyennneur d'ordre $M = 8$ (9 coefficients tous égaux à $1/9$).
2. L'appliquer à un signal bruité.
3. Comparer l'entrée et la sortie.
4. Tracer la réponse en fréquence.

🔗 Code MATLAB : Exercice MATLAB 2 – Filtre RIF moyennneur

```

1  %%
   =====
2  %  EXERCICE 2 : Filtre RIF moyennneur d'ordre M=8
3  %%
   =====

4  clear all; clc; close all;
5
6  %--- 1. Generation du signal d'entree bruite ---
7  N = 200;                % Nombre d'echantillons
8  n = 0:N-1;
9  Fe = 1000;              % Frequence d'echantillonnage (Hz)
10
11 % Signal utile : sinusoide basse frequence (50 Hz)
12 signal_utile = sin(2*pi*50/Fe * n);
13
14 % Bruit blanc gaussien (forte amplitude)
15 bruit = 0.8 * randn(1, N);
16
17 % Signal bruite = signal utile + bruit
18 x = signal_utile + bruit;
19
20 %--- 2. Conception du filtre RIF moyennneur (M=8) ---
21 M = 8;                  % Ordre du filtre
22 b = ones(1, M+1) / (M+1); % Coefficients : [1/9, 1/9, ...,
    1/9]

```

```

23 a = 1; % Denominateur = 1 (filtre RIF pur)
24
25 disp('=== FILTRE RIF MOYENNEUR ===');
26 fprintf('Ordre : M = %d\n', M);
27 fprintf('Nombre de coefficients : %d\n', M+1);
28 disp('Coefficients b :'); disp(b);
29
30 %--- 3. Application du filtre ---
31 y = filter(b, a, x); % Filtrage de x pour obtenir y
32
33 %--- 4. Visualisation temporelle ---
34 figure('Name','Exercice 2 - Filtre RIF','NumberTitle','off');
35
36 subplot(3,1,1);
37 plot(n, x, 'Color', [0.7 0.7 0.7], 'LineWidth', 1);
38 hold on;
39 plot(n, signal_utile, 'b--', 'LineWidth', 1.5);
40 xlabel('Echantillon n'); ylabel('Amplitude');
41 title('Signal d\'entree brute', 'FontSize', 12, 'FontWeight',
    'bold');
42 legend('Signal brute x[n]', 'Signal utile (reference)', '
    Location', 'best');
43 grid on;
44
45 subplot(3,1,2);
46 plot(n, y, 'r', 'LineWidth', 1.5);
47 hold on;
48 plot(n, signal_utile, 'b--', 'LineWidth', 1);
49 xlabel('Echantillon n'); ylabel('Amplitude');
50 title('Signal filtre (sortie RIF moyenneur)', 'FontSize', 12, '
    FontWeight', 'bold');
51 legend('Signal filtre y[n]', 'Signal utile (reference)', '
    Location', 'best');
52 grid on;
53
54 subplot(3,1,3);
55 [H, w] = freqz(b, a, 512);
56 plot(w/pi*Fe/2, 20*log10(abs(H)+1e-10), 'g', 'LineWidth', 2);
57 xlabel('Frequence (Hz)'); ylabel('|H| (dB)');
58 title('Reponse en frequence du filtre RIF', 'FontSize', 12, '
    FontWeight', 'bold');
59 grid on; xlim([0 Fe/2]);
60 yline(-3, 'r--', '-3 dB', 'LineWidth', 1.5);
61
62 sgtitle('Filtrage RIF Moyenneur (M=8)', 'FontSize', 14, '
    FontWeight', 'bold');
63
64 %--- 5. Calcul du RSB avant/apres ---

```

```

65 RSB_avant = 10*log10(sum(signal_utile.^2) / sum(bruit.^2));
66 erreur_apres = y - signal_utile;
67 RSB_apres = 10*log10(sum(signal_utile.^2) / sum(erreur_apres
    .^2));
68 fprintf('\n=== RAPPORT SIGNAL/BRUIT ===\n');
69 fprintf('RSB avant filtrage : %.1f dB\n', RSB_avant);
70 fprintf('RSB apres filtrage : %.1f dB\n', RSB_apres);
71 fprintf('Amelioration : %.1f dB\n', RSB_apres -
    RSB_avant);

```

☑ Correction – Exercice MATLAB 2

Résultats attendus :

- Le filtre RIF moyenneur d'ordre 8 est un **filtre passe-bas** avec une fréquence de coupure $f_c \approx F_e/(M+1) \approx 111$ Hz.
- Le RSB s'améliore typiquement de 8 à 10 dB.
- On observe un **retard** de $M/2 = 4$ échantillons (phase linéaire, propriété des filtres RIF symétriques).
- Les oscillations résiduelles sont les composantes du bruit qui ne sont pas filtrées (en-dessous de f_c).

8.3 Exercice MATLAB 3 : Conception d'un filtre RII de Butterworth

⚡ Exercice MATLAB 3 : Filtre passe-bas de Butterworth

Concevoir un filtre RII passe-bas de Butterworth d'ordre 4 avec une fréquence de coupure normalisée $\omega_c = 0,3\pi$ (soit $f_c = 150$ Hz pour $F_e = 1000$ Hz).

1. Utiliser la fonction `butter` de MATLAB.
2. Tracer la réponse en fréquence et le diagramme pôles-zéros.
3. Analyser la stabilité.
4. Appliquer le filtre au signal bruité de l'exercice précédent.

🔗 Code MATLAB : Exercice MATLAB 3 – Filtre RII Butterworth

```

1 %%
    =====
2 % EXERCICE 3 : Filtre RII passe-bas de Butterworth ordre 4
3 %%
    =====

```

```

4 clear all; clc; close all;
5
6 %--- 1. Parametres ---
7 Fe = 1000;          % Frequence d'echantillonnage (Hz)
8 fc = 150;           % Frequence de coupure (Hz)
9 wc = fc/(Fe/2);     % Frequence normalisee entre 0 et 1 (0.3)
10 ordre = 4;         % Ordre du filtre
11
12 %--- 2. Conception du filtre avec butter() ---
13 [b, a] = butter(ordre, wc, 'low');
14
15 disp('=== FILTRE BUTTERWORTH ORDRE 4 ===');
16 fprintf('Frequence de coupure normalisee : wc = %.2f\n', wc);
17 disp('Coefficients b (numérateur) :'); disp(b);
18 disp('Coefficients a (denominateur) :'); disp(a);
19
20 % Afficher la fonction de transfert
21 fprintf('Ordre du numerateur : %d\n', length(b)-1);
22 fprintf('Ordre du denominateur : %d\n', length(a)-1);
23
24 %--- 3. Analyse des poles ---
25 poles = roots(a);
26 zeros_H = roots(b);
27 disp('Poles :'); disp(poles);
28 disp('Module des poles :'); disp(abs(poles));
29 if all(abs(poles) < 1)
30     disp('>>> Filtre STABLE (tous poles dans le cercle unite)');
31 end
32
33 %--- 4. Visualisations ---
34 figure('Name','Exercice 3 - Butterworth','NumberTitle','off');
35
36 % Reponse en frequence (dB)
37 subplot(2,2,1);
38 [H, w] = freqz(b, a, 1024);
39 f = w / (2*pi) * Fe;      % Conversion en Hz
40 plot(f, 20*log10(abs(H)+1e-15), 'b', 'LineWidth', 2);
41 xlabel('Frequence (Hz)'); ylabel('|H(f)| (dB)');
42 title('Reponse en frequence', 'FontSize', 12, 'FontWeight', 'bold');
43 grid on; xlim([0 Fe/2]);
44 xline(fc, 'r--', 'fc = 150 Hz', 'LineWidth', 1.5);
45 yline(-3, 'g--', '-3 dB', 'LineWidth', 1.5);
46 ylim([-60 5]);
47
48 % Phase

```

```

49 subplot(2,2,2);
50 plot(f, unwrap(angle(H))*180/pi, 'r', 'LineWidth', 2);
51 xlabel('Frequence (Hz)'); ylabel('Phase (degrees)');
52 title('Reponse en phase', 'FontSize', 12, 'FontWeight', 'bold');
53 grid on; xlim([0 Fe/2]);
54
55 % Diagramme poles-zeros
56 subplot(2,2,3);
57 zplane(b, a);
58 title('Diagramme Poles Zeros', 'FontSize', 12, 'FontWeight', 'bold');
59 grid on;
60
61 % Reponse impulsionnelle
62 subplot(2,2,4);
63 [h, t] = impz(b, a, 60);
64 stem(t, h, 'filled', 'b', 'LineWidth', 1.2);
65 xlabel('Echantillon n'); ylabel('h[n]');
66 title('Reponse impulsionnelle', 'FontSize', 12, 'FontWeight', 'bold');
67 grid on;
68
69 sgtitle('Filtre Butterworth Passe-Bas Ordre 4', 'FontSize', 14, ...
70         'FontWeight', 'bold');
71
72 %--- 5. Application sur signal bruité ---
73 N = 300; n = 0:N-1;
74 signal_utile = sin(2*pi*50/Fe * n);
75 bruit = 0.8 * randn(1, N);
76 x = signal_utile + bruit;
77 y = filter(b, a, x);
78
79 figure('Name','Application Butterworth','NumberTitle','off');
80 subplot(2,1,1);
81 plot(n, x, 'Color',[0.7 0.7 0.7]); hold on;
82 plot(n, signal_utile, 'b--', 'LineWidth', 1.5);
83 title('Entree bruitée', 'FontSize', 12, 'FontWeight', 'bold');
84 xlabel('n'); ylabel('Amplitude'); grid on;
85 legend('x[n] bruité', 'signal utile', 'Location', 'best');
86
87 subplot(2,1,2);
88 plot(n, y, 'r', 'LineWidth', 1.5); hold on;
89 plot(n, signal_utile, 'b--', 'LineWidth', 1.5);
90 title('Sortie du filtre Butterworth', 'FontSize', 12, 'FontWeight', 'bold');
91 xlabel('n'); ylabel('Amplitude'); grid on;

```

```

92 legend('y[n] filtre', 'signal utile', 'Location', 'best');
93
94 sgtitle('Debruitage par Filtre Butterworth Ordre 4', 'FontSize
    ', 14, ...
95         'FontWeight', 'bold');

```

☑ Correction – Exercice MATLAB 3

Analyse des résultats :

- La commande `butter(4, 0.3, 'low')` génère un filtre Butterworth passe-bas d'ordre 4 avec $f_c = 150$ Hz.
- Les 4 pôles sont complexes conjugués, disposés en arc de cercle à l'intérieur du cercle unité $\Rightarrow$ **filtre stable**.
- À f_c , l'atténuation est exactement -3 dB (propriété du filtre de Butterworth).
- La pente de coupure est de -80 dB/décade (20 dB/décade $\times$ ordre 4).
- La réponse impulsionnelle est infinie (elle diminue exponentiellement) : c'est bien un **filtre RII**.
- Le filtre Butterworth est meilleur que le RIF moyenneur en termes d'atténuation dans la bande coupée.

8.4 Exercice MATLAB 4 : Analyse de la stabilité

⚡ Exercice MATLAB 4 : Étude de la stabilité d'un filtre RII

On considère les filtres suivants. Pour chacun, déterminer la stabilité et illustrer avec MATLAB :

- **Filtre A** : $H_A(z) = \frac{1}{1 - 1,5z^{-1} + 0,5z^{-2}}$ (pôles à 0,5 et 1)
- **Filtre B** : $H_B(z) = \frac{1}{1 - 0,9z^{-1}}$ (pôle à 0,9)
- **Filtre C** : $H_C(z) = \frac{1}{1 - 1,2z^{-1}}$ (pôle à 1,2)

Appliquer un échelon unité et observer les réponses.

⌘ Code MATLAB : Exercice MATLAB 4 – Étude de la stabilité

```

1 %%
    =====
2 %  EXERCICE 4 : Etude de la stabilite de filtres RII
3 %%
    =====

```

```

4 clear all; clc; close all;
5
6 %--- Definitions des filtres ---
7 % Filtre A : pole sur le cercle unite (marginalelement stable)
8 bA = [1]; aA = [1, -1.5, 0.5];
9
10 % Filtre B : pole strictement interieur (stable)
11 bB = [1]; aB = [1, -0.9];
12
13 % Filtre C : pole exterieur au cercle (instable)
14 bC = [1]; aC = [1, -1.2];
15
16 %--- Verification analytique ---
17 filtres = {'A', bA, aA}, {'B', bB, aB}, {'C', bC, aC}';
18
19 for i = 1:3
20     nom = filtres{i}{1};
21     b_f = filtres{i}{2};
22     a_f = filtres{i}{3};
23     poles = roots(a_f);
24     fprintf('\n--- Filtre %s ---\n', nom);
25     fprintf('Poles : '); disp(poles);
26     fprintf('Modules : '); disp(abs(poles));
27     if all(abs(poles) < 1)
28         fprintf('>>> STABLE\n');
29     elseif all(abs(poles) <= 1)
30         fprintf('>>> MARGINALEMENT STABLE\n');
31     else
32         fprintf('>>> INSTABLE\n');
33     end
34 end
35
36 %--- Reponses a un echelon ---
37 N = 50;
38 u = ones(1, N); % Echelon unite
39
40 yA = filter(bA, aA, u);
41 yB = filter(bB, aB, u);
42 yC = filter(bC, aC, u);
43
44 % Saturer le filtre C pour l'affichage
45 yC_affiche = min(max(yC, -100), 100);
46
47 figure('Name','Exercice 4 - Stabilite','NumberTitle','off');
48
49 subplot(3,2,1);
50 zplane(bA, aA); title('Filtre A - Poles/Zeros', 'FontWeight',

```

```

        'bold');
51
52 subplot(3,2,2);
53 stem(0:N-1, yA, 'filled', 'b'); grid on;
54 title('Filtre A - Reponse echelon (Marg. stable)', 'FontWeight',
        'bold');
55 xlabel('n'); ylabel('y[n]');
56
57 subplot(3,2,3);
58 zplane(bB, aB); title('Filtre B - Poles/Zeros', 'FontWeight',
        'bold');
59
60 subplot(3,2,4);
61 stem(0:N-1, yB, 'filled', 'g'); grid on;
62 title('Filtre B - Reponse echelon (STABLE)', 'FontWeight', 'bold');
63 xlabel('n'); ylabel('y[n]');
64
65 subplot(3,2,5);
66 zplane(bC, aC); title('Filtre C - Poles/Zeros', 'FontWeight',
        'bold');
67
68 subplot(3,2,6);
69 stem(0:N-1, yC_affiche, 'filled', 'r'); grid on;
70 title('Filtre C - Reponse echelon (INSTABLE !)', 'FontWeight',
        'bold');
71 xlabel('n'); ylabel('y[n] (sature)');
72
73 sgtitle('Comparaison Stabilite des Filtres RII', 'FontSize',
        14, ...
        'FontWeight', 'bold');
74
75
76 fprintf('\n=== CONCLUSION ===\n');
77 fprintf('Filtre A : pole en z=1 (bord du cercle) ->
        oscillations constantes\n');
78 fprintf('Filtre B : pole en z=0.9 (interieur) -> converge
        vers une valeur finie\n');
79 fprintf('Filtre C : pole en z=1.2 (exterieur) -> diverge
        vers l infini\n');

```

✓ Correction – Exercice MATLAB 4

Résultats et interprétation :

- **Filtre A** ($p = 1$ et $p = 0,5$) : Un pôle est exactement sur le cercle unité ($|p| = 1$). La réponse à l'échelon est une **rampe** (croissance linéaire). C'est un filtre **marginalelement stable**.
- **Filtre B** ($p = 0,9$) : Le pôle est à l'intérieur du cercle ($|p| = 0,9 < 1$). La

réponse à l'échelon converge vers $1/(1 - 0,9) = 10$. Filtre **stable**.

- **Filtre C** ($p = 1,2$) : Le pôle est à l'extérieur ($|p| = 1,2 > 1$). La réponse diverge exponentiellement. Filtre **instable** : à éviter absolument !

8.5 Exercice MATLAB 5 : Comparaison RIF vs RII

⚡ Exercice MATLAB 5 : Filtre passe-bas RIF vs RII

Concevoir deux filtres passe-bas ayant les mêmes spécifications :

- Fréquence de coupure : $f_c = 200$ Hz pour $F_e = 1000$ Hz
- Atténuation en bande coupée : > 40 dB

Comparer l'ordre nécessaire et les performances.

📄 Code MATLAB : Exercice MATLAB 5 – RIF vs RII

```

1  %%
   =====
2  %  EXERCICE 5 : Comparaison RIF vs RII
3  %%
   =====

4  clear all; clc; close all;
5
6  Fe = 1000; fc = 200;
7  wc = fc / (Fe/2); % Frequence normalisee
8
9  %--- 1. Filtre RII de Butterworth ---
10 ordre_RII = 4;
11 [b_RII, a_RII] = butter(ordre_RII, wc, 'low');
12
13 %--- 2. Filtre RIF (fenetrage de Hamming) ---
14 % Pour avoir ~40 dB d'attenuation avec une fenetre de Hamming
   :
15 ordre_RIF = 30; % Ordre empirique (a ajuster)
16 b_RIF = fir1(ordre_RIF, wc, 'low', hamming(ordre_RIF+1));
17 a_RIF = 1;
18
19 fprintf('=== COMPARAISON RIF vs RII ===\n');
20 fprintf('Ordre RII : %d (nombre de coefficients : %d)\n',
   ...
21         ordre_RII, length(b_RII)+length(a_RII)-1);
22 fprintf('Ordre RIF : %d (nombre de coefficients : %d)\n',
   ...

```

```

23         ordre_RIF, length(b_RIF));
24
25 %--- 3. Reponses en frequence ---
26 [H_RII, w] = freqz(b_RII, a_RII, 1024);
27 [H_RIF, ~] = freqz(b_RIF, a_RIF, 1024);
28 f = w / (2*pi) * Fe;
29
30 figure('Name','Exercice 5 - RIF vs RII','NumberTitle','off');
31
32 subplot(2,2,1);
33 plot(f, 20*log10(abs(H_RII)+1e-15), 'b', 'LineWidth', 2); hold
    on;
34 plot(f, 20*log10(abs(H_RIF)+1e-15), 'r--', 'LineWidth', 2);
35 xlabel('Frequence (Hz)'); ylabel('|H| (dB)');
36 title('Reponse en frequence (dB)', 'FontWeight', 'bold');
37 legend('RII Butterworth ord.4', 'RIF Hamming ord.30', '
    Location', 'best');
38 grid on; xlim([0 Fe/2]); ylim([-80 5]);
39 xline(fc, 'k--'); yline(-40, 'g--', '-40 dB');
40
41 subplot(2,2,2);
42 plot(f, unwrap(angle(H_RII))*180/pi, 'b', 'LineWidth', 2);
    hold on;
43 plot(f, unwrap(angle(H_RIF))*180/pi, 'r--', 'LineWidth', 2);
44 xlabel('Frequence (Hz)'); ylabel('Phase (degrees)');
45 title('Reponse en phase', 'FontWeight', 'bold');
46 legend('RII (non lineaire)', 'RIF (lineaire)', 'Location', '
    best');
47 grid on; xlim([0 Fe/2]);
48
49 subplot(2,2,3);
50 [h_RII, t] = impz(b_RII, a_RII, 60);
51 [h_RIF, ~] = impz(b_RIF, a_RIF, 60);
52 stem(t, h_RII, 'b', 'filled', 'LineWidth', 1.2); hold on;
53 stem(t, h_RIF, 'r', 'LineWidth', 1.2);
54 xlabel('n'); ylabel('h[n]');
55 title('Reponse impulsionnelle', 'FontWeight', 'bold');
56 legend('RII', 'RIF');
57 grid on;
58
59 subplot(2,2,4);
60 % Diagramme poles-zeros RII
61 zplane(b_RII, a_RII);
62 title('Poles-Zeros du filtre RII', 'FontWeight', 'bold');
63
64 sgtitle('Comparaison Filtre RIF vs RII', 'FontSize', 14, '
    FontWeight', 'bold');
65

```

```

66 fprintf('\n=== CONCLUSIONS ===\n');
67 fprintf('1. Phase lineaire : RIF oui, RII non\n');
68 fprintf('2. Ordre necessaire : RIF beaucoup plus eleve que RII\n');
69 fprintf('3. Stabilite : RIF toujours stable, RII a verifier\n');
70 fprintf('4. Cout calcul : RIF plus couteux (plus de coefficients)\n');

```

9 Récapitulatif et Aide-mémoire

📖 AIDE-MÉMOIRE : Filtres Numériques

1. Transformée en Z :

$$X(z) = \sum_{n=-\infty}^{+\infty} x[n]z^{-n}, \quad \text{Retard : } x[n-k] \leftrightarrow z^{-k}X(z)$$

2. Équation de récurrence générale :

$$y[n] = \sum_{k=0}^M b_k x[n-k] - \sum_{k=1}^N a_k y[n-k]$$

3. Fonction de transfert :

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\sum_{k=0}^M b_k z^{-k}}{1 + \sum_{k=1}^N a_k z^{-k}}$$

4. **Filtre RIF** : Dénominateur = 1 (pas de a_k , pas de rétroaction). Toujours stable.

5. **Filtre RII** : Dénominateur $\neq 1$ (présence de a_k , rétroaction). Peut être instable.

6. **Stabilité** : Tous les pôles doivent satisfaire $|p_i| < 1$ (strictement à l'intérieur du cercle unité).

7. **Passage $H(z) \rightarrow$ récurrence** : Produit en croix, puis $z^{-k}Y(z) \rightarrow y[n-k]$.

8. **Passage récurrence $\rightarrow H(z)$** : Transformée en Z de chaque terme, puis $H = Y(z)/X(z)$.

! Commandes MATLAB Essentielles

<code>filter(b,a,x)</code>	Appliquer le filtre de coefficients b,a au signal x
<code>freqz(b,a)</code>	Tracer la réponse en fréquence
<code>zplane(b,a)</code>	Diagramme pôles-zéros
<code>impz(b,a)</code>	Réponse impulsionnelle
<code>roots(a)</code>	Trouver les pôles (racines de a)
<code>butter(N,wc)</code>	Filtre RII Butterworth d'ordre N
<code>fir1(N,wc)</code>	Filtre RIF par fenêtrage d'ordre N
<code>abs(poles) < 1</code>	Tester la stabilité